FRI Natural language processing course 2025/2026
Project report

University *of Ljubljana*
Faculty *of Computer and Information Science*

RAG-based Grafana assistant

Mojca Kompara, Miha Meglič, and Andraž Sovinec

Abstract

Grafana's extensive features and capabilities are supported by vast documentation, making it difficult for users to quickly locate specific information. This report presents the development of an AI-powered assistant designed specifically for official Grafana documentation to help system administrators and developers resolve queries efficiently. Utilizing a Retrieval-Augmented Generation (RAG) architecture running locally via Ollama, the system processes documentation chunks with LlamaIndex and stores them in a vector database for semantic retrieval. Two iterations of the assistant were tested: an initial fixed-length chunking model and a final version using file-direct metadata extraction. Performance analysis using 20 complex technical queries demonstrated that the RAG-based assistant significantly outperformed a standard Llama 3.1 8B model in both accuracy and helpfulness. The results suggest that deploying a specialized RAG framework offers a more precise, context-aware alternative to general AI tools, reducing setup and troubleshooting time for Grafana users.

Keywords

Retrieval-Augmented Generation (RAG), Grafana, Large Language Models (LLM), Technical Documentation, AI Assistant, LlamaIndex, Natural Language Processing.

Advisors: Slavko Žitnik

Introduction

Grafana is a widely used open-source platform for data visualization, monitoring, and analysis. Because it offers many features and supports a wide range of use cases, its documentation is extensive. As a result, users may sometimes find it difficult to navigate efficiently. The goal of our project is to build an AI assistant that helps users access relevant parts of the documentation more easily. The assistant will retrieve and present information from the documentation based on the user's query. This could significantly reduce the time needed to solve specific problems, since users would no longer have to manually search through large sections of documentation. The target users are system administrators, developers, and casual Grafana users.

There are already existing solutions for this problem. Grafana includes a keyword-based search function, and it also provides its own AI assistant. However, that assistant is designed to provide more general information about Grafana, while our solution will focus specifically on the official documentation. Because of this narrower focus, our assistant should be able to deliver more precise and relevant answers to documentation-related questions.

The primary motivation for this project is to reduce the time it takes users to set up or learn something new when using grafana and provide them with instant, reliable answers. We are going to use the RAG approach. With this approach, the system first retrieves the most relevant sections of the documentation in response to a user query and then uses a language model to generate an answer based on those retrieved sources. This allows the assistant to provide responses that are more accurate, context-aware, and grounded in the documentation itself. As a result, the system can reduce irrelevant answers and help users find reliable information more quickly.

Related Work

In higher education, RAG-based chatbots have been successfully applied to student support. For instance, the College RAG Chatbot ingests college brochures, FAQs, and guides to answer admissions and academic queries accurately from verified sources [1]. Our AI assistant will not be used for student support, but this working chatbot could still give us a lot of helpful insight for making it.

To address complex queries over large document collections, Lee [2] proposes a RAG framework that decomposes multihop questions into single-hop subquestions using an LLM.

Proposed Data

For this project, we will use web-accessible documentation from the official Grafana webpage (https://grafana.com/docs/).

Initial implementation

We decided to use the Retrieval-Augmented Generation (RAG) approach to build our chatbot. This approach allows the model to generate answers based on relevant external documentation instead of relying only on its pre-trained knowledge. For our implementation, we used Ollama, an open-source tool that makes it easier to run large language models locally. To prepare the documentation for the RAG pipeline, we used the Python library LlamaIndex. This library helped us load the Grafana documentation files, process their contents, and split them into smaller chunks that could be stored and retrieved efficiently. After chunking the documentation, we embedded the chunks, stored them in a vector database, and retrieved the most relevant ones for each query before passing them to the language model.

Performance analysis

These results should be interpreted as an initial evaluation, since the benchmark contains only 20 queries and the quality scores are based on subjective human judgment.

The results varied in both quality and response time. The response times ranged from around 12 seconds up to 67 seconds. The quality of the responses was rated based on the subjective opinions of two separate evaluators. Most of the responses were good, but a few were very poor. Queries with shorter response times were usually those that required short and simple answers, while queries with longer response times were generally those that asked how to set up something and therefore required a description of a full process. The queries used for testing are listed below, and the metrics from the tests are shown in table 1.

Queries used for testing

1. ”What is Grafana used for?”
2. ”How do I create a new dashboard?”
3. ”Explain how to set up alerts.”
4. ”What data sources does Grafana support?”
5. ”How can I share a dashboard with others?”
6. ”Describe the process to install plugins.”
7. ”How do I manage user permissions?”
8. ”What is the default port for Grafana?”
9. ”How to upgrade Grafana to the latest version?”
10. ”Where are Grafana logs stored?”
11. ”How do I change the Grafana admin password?”
12. ”How do I add a Prometheus data source?”
13. ”Can Grafana send alert notifications?”
14. ”How do I back up Grafana?”
15. ”Where is the Grafana configuration file located?”
16. ”How do I invite another user to Grafana?”
17. ”How do I install a panel plugin?”
18. ”Can Grafana connect to Elasticsearch?”
19. ”How do I export a dashboard?”
20. ”What happens if Grafana cannot find a data source?”

Table 1. Model analysis

No. of question	Response time [s]	Rating
1	23.42	4/5
2	44.40	4/5
3	67.17	5/5
4	24.55	5/5
5	26.29	5/5
6	52.27	5/5
7	31.61	1/5
8	12.59	5/5
9	50.46	2/5
10	29.37	0/5
11	43.86	3/5
12	28.45	1/5
13	22.25	5/5
14	47.19	5/5
15	28.26	5/5
16	42.38	5/5
17	21.07	5/5
18	15.33	5/5
19	29.75	5/5
20	15.29	2/5
Average	32.80	3.85

Problematic cases

One case that we could consider problematic is the query “Explain how to set up alerts,” since it had the longest response time. It is true that this query requires a description of an entire process, but 67.17 seconds still feels too long. One possible reason for the long response time is that the model may have needed to combine information from multiple chunks in order to generate a response, instead of relying mainly on the best-matching chunk. The problem here is mainly the efficiency of collecting the information.

There are also some problematic cases where the model misses the main point of the question. Examples include the queries "Where are Grafana logs stored?" and "How do I manage user permissions?". For the first query, the response described ways to store logs outside of Grafana instead of giving the location of Grafana logs. For the second query, the response described the different roles that can be assigned to users and the rules that should be followed, but it did not provide actual information on how to set those permissions. One possible explanation is that the retrieval step favored chunks containing matching keywords, even when those chunks did not answer the full intent of the question.

Future directions and ideas

One possible way to improve performance would be to clean the Grafana documentation by removing duplicates and organizing the source documents better. This can have a big effect on RAG systems, because cleaner and better-structured data usually leads to more accurate retrieval. We could also experiment with different chunking strategies to see whether any of them works significantly better than the others. For example, some strategies may preserve context better, while others may improve retrieval precision.

Another way to improve performance is to improve retrieval. This could be achieved by using a hybrid retrieval approach, which combines lexical and vector methods. Such an approach could help the system retrieve both exact keyword matches and semantically similar content. We could also try adapting retrieval to be query-specific, or additionally reordering the chunks after the initial retrieval to improve the quality of the final context given to the model.

The final idea for improvement is to try out different models and compare their performance. Some models might be better suited to this specific task than others, especially when dealing with technical documentation such as Grafana docs. In addition to answer quality, we could also compare the models based on speed, consistency, and hardware requirements.

Final implementation

For the final implementation we made some improvements to the previous version. The initial implementation was making chunks based on text length, but for the newer version no longer uses fixed length-based chunking. The other difference is that we get the metadata directly from the files now.

Performance analysis

These results were obtained from a test similar to the initial evaluation. We compared the results to the Llama 3.1 8B model [3] to see how it compares to a general AI model. We also made some more complicated queries, since the ones in the initial test were too simplistic. The results are displayed in the table 2 in appendix.

Queries used for testing

1. "What is Grafana used for?"
2. "How do I create a new dashboard in Grafana?"
3. "Explain how to set up alerts in Grafana."
4. "What data sources does Grafana support?"
5. "How can I share a dashboard with others in Grafana?"
6. "Describe the process to install plugins in Grafana."
7. "How do I manage user permissions in Grafana?"
8. "What is the default port for Grafana?"
9. "How to upgrade Grafana to the latest version?"
10. "Where are Grafana logs stored?"
11. "A team wants to migrate Grafana to a new server, keep existing dashboards, preserve user access, and avoid breaking plugins. What should they back up and reconfigure?"
12. "An administrator upgraded Grafana and now alerts are not firing, some dashboards show missing data sources, and users report access issues. What should be checked first and in what order?"
13. "How would you set up Grafana for a team where developers can edit dashboards, managers can only view them, and alert notifications must still reach both groups when needed?"
14. "If a dashboard must be shared externally but the underlying data source and editing permissions should remain restricted, what Grafana features or settings are relevant?"
15. "What steps would be involved in setting up a completely new Grafana instance with a Prometheus data source, alerting, user access control, and a shareable dashboard?"
16. "What files and directories are important when troubleshooting Grafana startup problems, especially when checking logs and configuration?"
17. "How do I troubleshoot a situation where alert rules exist but notifications are not being sent in Grafana?"

18. "How can I configure Grafana so that only certain users can access a specific dashboard while others can still use the rest of the system?"

19. "If I want to upgrade Grafana safely on a production system, what preparation and backup steps should I perform first?"

20. "How do I install a plugin, verify that Grafana recognizes it correctly, and troubleshoot the issue if it does not appear in the UI?"

Each model is rated on:

- **Accuracy**: is the information correct and Grafana-specific?
- **Helpfulness**: does it actually answer the user's question with the usable detail?

Our model performed overall better than the llama model. The answers were generally more helpful but more importantly, they were more accurate and Grafana specific in most cases. The few outliers where the Llama 3.1 8B model gave a better response were all on the simpler queries.

Problematic cases

Here we will cover the cases where our model performed worse than the Llama 3.1 8B model. In case 1, the Llama model gave a borader answer. It had more information because it is trained on a much broader dataset. In case 4, our model listed less supported data sources. There are probably some compatible data sources that are not explicitly listed in the documentation. In case 8, the query was "What is the default port for Grafana?". Our model responded with only the number 3000. The answer is correct but it could just add a few more words. A good example would be "The default port for Grafana is 3000." or something similar. The remaining 3 cases probably have similar reasons, where some information from outside of the documentation was crucial to the answer.

Conclusion

This project showed that a RAG-based assistant can be an effective tool for navigating extensive technical documentation. The final system produced answers that were generally more accurate, more relevant, and more helpful than those of a standalone general-purpose LLM, especially for complex queries. Although there is still room for improvement in retrieval quality, response speed, and evaluation methodology, the results indicate that a domain-specific RAG approach is a practical solution for supporting Grafana users in real troubleshooting and setup tasks.

References

[1] Deep Shah. College rag chatbot. https://github.com/DeepShah1406/College_RAG_Chatbot, 2024. Accessed: March 20, 2026.

[2] Seokgi Lee. Transforming questions and documents for semantically aligned retrieval-augmented generation. *arXiv preprint arXiv:2508.09755*, 2025.

[3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Aiesha Al-Dahle, Adam Letman, Ankur Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Arram Sravankumar, Artem Korenev, Austin Hinsvark, Bhargav Rao, Brandon Zhang, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Appendix

Table 2. Model analysis

	RAG		Direct			Score		Time (s)	
#	Acc.	Help.	Acc.	Help.	Winner	RAG	Dir.	RAG	Dir.
1	◐	◐	✓	◐	Direct	4/5	5/5	2.695	1.017
2	◐	◐	◐	◐	Direct	2/5	2/5	2.832	1.132
3	✓	✓	◐	◐	RAG	5/5	2/5	3.787	3.620
4	◐	×	✓	✓	Direct	3/5	4/5	1.622	0.990
5	✓	✓	◐	×	RAG	5/5	2/5	4.798	0.889
6	◐	◐	✓	✓	Direct	3/5	5/5	5.063	1.705
7	✓	✓	◐	◐	RAG	5/5	3/5	3.007	1.443
8	✓	◐	✓	✓	Direct	5/5	5/5	2.082	0.323
9	✓	◐	◐	◐	RAG	5/5	4/5	5.214	1.833
10	◐	×	×	×	RAG	5/5	2/5	1.589	1.295
11	◐	◐	✓	✓	Direct	3/5	4/5	2.813	1.707
12	◐	◐	✓	✓	Direct	3/5	4/5	3.379	1.682
13	✓	✓	◐	◐	RAG	5/5	4/5	4.866	2.847
14	✓	✓	◐	◐	RAG	5/5	4/5	4.482	2.173
15	✓	✓	◐	◐	RAG	5/5	4/5	5.429	3.380
16	✓	✓	◐	◐	RAG	5/5	4/5	3.805	2.553
17	✓	✓	◐	◐	RAG	5/5	2/5	4.781	1.926
18	✓	✓	×	×	RAG	5/5	4/5	4.154	3.058
19	✓	✓	◐	◐	RAG	5/5	5/5	3.458	1.783
20	✓	✓	◐	✓	RAG	5/5	3/5	5.914	3.518